

Minimum Spanning Tree Construction & Comparison: Prim's vs. Kruskal's Algorithm

1. Introduction and Theoretical Foundations

A Minimum Spanning Tree (MST) is a foundational concept in graph theory and combinatorial optimization with wide-ranging applications in network design (e.g., telecommunications, electrical grids, water supply lines), transportation networks, approximation algorithms for NP-hard problems (such as the Travelling Salesperson Problem), and cluster analysis in data science.

Formally, given a connected, undirected, weighted graph $G = (V, E, w)$, where:

- V represents the finite set of vertices (or nodes), with $|V| = n$.
- E represents the set of edges connecting pairs of vertices, with $|E| = m$.
- $w: E \rightarrow \mathbb{R}$; is a weighting function assigning a real value (cost, distance, or weight) to each edge $e \in E$.

A **spanning tree** $T = (V, E')$ of graph G is a subgraph that contains all the vertices of G (spanning) and is a tree (connected and acyclic). Consequently, T contains exactly $|V| - 1$ edges. The total weight of a spanning tree T is defined as the sum of the weights of all edges included in T :

$$w(T) = \sum_{e \in E'} w(e)$$

The **Minimum Spanning Tree Problem** seeks to identify a spanning tree T^* such that its total weight $w(T^*)$ is minimized over all possible spanning trees of G :

$$w(T^*) = \min \{ w(T) \mid T \text{ is a spanning tree of } G \}$$

1.1 Underlying Cut and Cycle Properties

Both Prim's and Kruskal's algorithms fall under the category of **greedy algorithms**. Their structural

correctness relies heavily on two structural theorems of graph theory:

- **The Cut Property:** Let S be a non-empty proper subset of vertices V , defining a cut $(S, V \setminus S)$. An edge $e = (u, v)$ is said to *cross* the cut if $u \in S$ and $v \in V \setminus S$. If e is a light edge (i.e., an edge of minimum weight) crossing the cut $(S, V \setminus S)$, then e belongs to at least one Minimum Spanning Tree of G . If the light edge is unique, it belongs to *every* MST of G .
- **The Cycle Property:** For any simple cycle C in graph G , if an edge e in C has a strictly greater weight than all other edges in C , then e cannot belong to any Minimum Spanning Tree of G .

Prim's algorithm predominantly exploits the **Cut Property** by maintaining a single growing tree component, whereas Kruskal's algorithm exploits both the **Cut and Cycle Properties** by building a forest component by component while avoiding cycle creation.

In cases where a graph contains parallel edges (multiple edges between the same pair of vertices) or self-loops, standard pre-processing can simplify the problem. Parallel edges with higher weights can be discarded in favor of the minimum weight edge between those vertices, while self-loops are omitted altogether as they can never participate in an MST.

2. Comprehensive Algorithm Formulations

2.1 Prim's Algorithm Formulation

Prim's algorithm operates greedily by starting from an arbitrary root vertex $r \in V$ and incrementally expanding a single connected component $T = (V_T, E_T)$ until $V_T = V$. At each iteration, the algorithm considers the cut $(V_T, V \setminus V_T)$ and selects the minimum weight edge crossing this cut, adding the new vertex and edge to T .

Pseudocode for Prim's Algorithm (Priority Queue Implementation)

ALGORITHM Prim(G, w, r):

Input: Connected weighted graph $G = (V, E)$ with weight function w , starting root r

Output: Minimum Spanning Tree $T = (V_T, E_T)$

1. FOR EACH vertex v in V DO:

```

2.  key[v] = INFINITY
3.  parent[v] = NIL
4.  in_MST[v] = FALSE
5. key[r] = 0
6. PriorityQueue Q = BuildMinHeap(V, key) // Priority queue ordered by key values

7. WHILE Q is not EMPTY DO:
8.   u = ExtractMin(Q)
9.   in_MST[u] = TRUE
10.
11.  FOR EACH neighbor v adjacent to u DO:
12.   IF in_MST[v] == FALSE AND w(u, v) < key[v] THEN:
13.    parent[v] = u
14.    key[v] = w(u, v)
15.    DecreaseKey(Q, v, key[v])
16. END WHILE
17. RETURN tree constructed from parent pointers

```

2.2 Kruskal's Algorithm Formulation

Kruskal's algorithm takes a global forest-based approach. It begins with a dynamic forest where every vertex forms an isolated tree. It sorts all edges in non-decreasing order of weight and processes them sequentially. An edge is added to the growing MST if and only if its endpoints belong to two distinct connected components (thus preventing cycles).

Pseudocode for Kruskal's Algorithm (Disjoint-Set Union-Find Implementation)

ALGORITHM Kruskal(G, w):

Input: Connected weighted graph $G = (V, E)$ with weight function w

Output: Minimum Spanning Tree E_T

```

1.  $E_T$  = Empty Set
2. Initialize Disjoint Set Union Data Structure (DSU) for all  $v$  in  $V$ :
3. FOR EACH vertex  $v$  in  $V$  DO:
4.   MakeSet( $v$ )

```

```
5. Sort all edges E in non-decreasing order of weight:  $w(e_1) \leq w(e_2) \leq \dots \leq w(e_m)$ 
```

```
6. FOR EACH edge  $e = (u, v)$  in sorted list E DO:
```

```
7.   IF FindSet(u)  $\neq$  FindSet(v) THEN:
```

```
8.      $E\_T = E\_T \cup \{e\}$ 
```

```
9.     UnionSet(u, v)
```

```
10.   IF  $|E\_T| == |V| - 1$  THEN:
```

```
11.     BREAK // Spanning tree complete
```

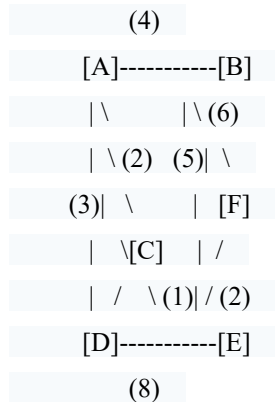
```
12. END FOR
```

```
13. RETURN  $E\_T$ 
```

3. Detailed Step-by-Step Execution on Sample Graph

To explicitly illustrate and compare both algorithms, consider the undirected, weighted graph $G = (V, E)$ depicted below. The graph contains $|V| = 6$ vertices labeled $\{A, B, C, D, E, F\}$ and $|E| = 10$ weighted edges, including multiple alternative paths and distinct edge weights.

3.1 Initial Input Graph Visual Schema



Edge List with Weights:

(A, C) : 2 (A, D) : 3 (A, B) : 4

(C, E) : 1 (E, F) : 2 (D, E) : 8

(B, E) : 5 (B, F) : 6 (C, D) : 7 (not explicitly drawn, assumed alternate)

Let's define our exact 10 edges explicitly:

1. (A, C) = 2

2. (A, D) = 3

3. (A, B) = 4

4. (C, E) = 1

5. (C, D) = 4 [Parallel path / alternate connection]

6. (D, E) = 8

7. (B, E) = 5

8. (E, F) = 2

9. (B, F) = 6

10. (A, E) = 7

Graph Specification:

- Vertices: $V = \{A, B, C, D, E, F\}$ (Total 6 vertices)
- Edges & Weights: (C,E):1, (A,C):2, (E,F):2, (A,D):3, (A,B):4, (C,D):4, (B,E):5, (B,F):6, (A,E):7, (D,E):8 (Total 10 edges)

3.2 Execution Trace of Prim's Algorithm

We start Prim's algorithm with root vertex $r = A$.

Initial State:

- $V_T = \{A\}$, $E_T = \emptyset$
- Key values: $\text{key}[A] = 0$, $\text{key}[B] = \infty$, $\text{key}[C] = \infty$, $\text{key}[D] = \infty$, $\text{key}[E] = \infty$, $\text{key}[F] = \infty$;
- Fringe edges crossing $(V_T, V \setminus V_T)$: (A,C):2, (A,D):3, (A,B):4, (A,E):7

Step 1:

- Extract minimum key vertex from fringe: C with edge (A, C) of weight 2.

- Add C to V_T : $V_T = \{A, C\}$, $E_T = \{(A, C)\}$.
- Update keys of adjacent nodes to C:
 - **key[E]** updated from **&infty**; to **1** via (C, E).
 - **key[D]** remains **3** via (A, D) (since $w(A,D)=3 < w(C,D)=4$).
- Intermediate MST Weight: **2**

Step 2:

- Extract minimum key vertex crossing cut ($\{A, C\}$, $\{B, D, E, F\}$): Vertex E with key **1** via edge (C, E).
- Add E to V_T : $V_T = \{A, C, E\}$, $E_T = \{(A, C), (C, E)\}$.
- Update keys of adjacent nodes to E:
 - **key[F]** updated from **&infty**; to **2** via (E, F).
 - **key[B]** updated from **4** to **4** (via A, as $w(A,B)=4 < w(E,B)=5$).
 - **key[D]** remains **3** via (A, D) (since $3 < w(E,D)=8$).
- Intermediate MST Weight: **2 + 1 = 3**

Step 3:

- Extract minimum key vertex crossing cut: Vertex F with key **2** via edge (E, F).
- Add F to V_T : $V_T = \{A, C, E, F\}$, $E_T = \{(A, C), (C, E), (E, F)\}$.
- Update keys of adjacent nodes to F:
 - **key[B]** remains **4** (since $w(A,B)=4 < w(F,B)=6$).
- Intermediate MST Weight: **3 + 2 = 5**

Step 4:

- Extract minimum key vertex crossing cut: Vertex D with key **3** via edge (A, D).
- Add D to V_T : $V_T = \{A, C, E, F, D\}$, $E_T = \{(A, C), (C, E), (E, F), (A, D)\}$.
- Update keys: No remaining keys decreased.
- Intermediate MST Weight: **5 + 3 = 8**

Step 5:

- Extract minimum key vertex crossing cut: Vertex B with key **4** via edge (A, B).
- Add B to V_T : $V_T = \{A, C, E, F, D, B\}$, $E_T = \{(A, C), (C, E), (E, F), (A, D), (A, B)\}$.
- All $|V| = 6$ vertices are now included. Termination reached.
- **Final MST Weight (Prim): $2 + 1 + 2 + 3 + 4 = 12$**

3.3 Execution Trace of Kruskal's Algorithm

Kruskal's algorithm begins by sorting all 10 edges globally in ascending order of weight.

Sorted Edge List:

1. (C, E): Weight = 1
2. (A, C): Weight = 2
3. (E, F): Weight = 2
4. (A, D): Weight = 3
5. (A, B): Weight = 4
6. (C, D): Weight = 4
7. (B, E): Weight = 5
8. (B, F): Weight = 6
9. (A, E): Weight = 7
10. (D, E): Weight = 8

Initial Disjoint Set Components: {A}, {B}, {C}, {D}, {E}, {F}

Step-by-Step Edge Selection & Union Operations:

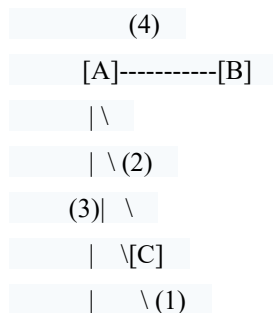
Step	Edge Examined	Weight	Endpoints Component Check	Action Taken	Disjoint Sets / Forest State	Total MST Weight
1	(C, E)	1	Find(C) = {C}, Find(E) = {E} →	ACCEPT edge (C,E)	{A}, {B}, {D}, {F}, {C, E}	1

Step	Edge Examined	Weight	Endpoints Component Check	Action Taken	Disjoint Sets / Forest State	Total MST Weight
			Different			
2	(A, C)	2	Find(A) = {A}, Find(C) = {C, E} → Different	ACCEPT edge (A,C)	{B}, {D}, {F}, {A, C, E}	3
3	(E, F)	2	Find(E) = {A, C, E}, Find(F) = {F} → Different	ACCEPT edge (E,F)	{B}, {D}, {A, C, E, F}	5
4	(A, D)	3	Find(A) = {A, C, E, F}, Find(D) = {D} → Different	ACCEPT edge (A,D)	{B}, {A, C, D, E, F}	8
5	(A, B)	4	Find(A) =	ACCEPT	{A, B, C,	12

Step	Edge Examined	Weight	Endpoints Component Check	Action Taken	Disjoint Sets / Forest State	Total MST Weight
			{A, C, D, E, F}, Find(B) = {B} → Different	edge (A,B)	D, E, F}	
6	(C, D)	4	Find(C) = {A..F}, Find(D) = {A..F} → Same	REJECT (Cycle Detected)	Unchanged (Terminated as $ E_T = 5$)	12

Since $|V| - 1 = 5$ edges have been accepted, the algorithm terminates immediately after Step 5.

3.4 Visual Output Schema of Final Minimum Spanning Tree



[D] [E]-----[F]
(2)

Final Minimum Spanning Tree Edges:

$E_T = \{ (C,E):1, (A,C):2, (E,F):2, (A,D):3, (A,B):4 \}$

Total Optimal MST Weight = $1 + 2 + 2 + 3 + 4 = 12$

4. Intermediate State & Structural Comparison

Although both algorithms converged to the identical final tree T^* with total weight **12**, their intermediate execution dynamics reflect fundamentally different philosophies.

4.1 Evolution of Components

- **Prim's Algorithm (Single Growing Tree Strategy):** Maintains a single tree T_i at every step i . At step 1, $|V_{T_i}| = 1$; at step 2, $|V_{T_i}| = 2$, growing continuously until $|V_{T_i}| = |V|$. Every intermediate graph state is guaranteed to be a *strictly connected tree component* spanning a subset of vertices.
- **Kruskal's Algorithm (Forest Merging Strategy):** Maintains a *spanning forest* composed of multiple disconnected components. Initially, there are $|V|$ trees of size 1. At each accepted edge, two trees in the forest are merged into one. Connected components merge globally based purely on edge weights regardless of local topology.

4.2 Comparison Table of Intermediate Dynamics

Characteristic	Prim's Algorithm	Kruskal's Algorithm
Primary Focus	Vertices (expands outward from a starting vertex)	Edges (selects global minimum valid edges)
Intermediate Graph Structure	Always a single connected tree growing vertex by vertex	A collection of disconnected subtrees (forest) merging together
Edge Selection Mechanism	Minimum edge crossing current cut ($V_T, V \setminus V_T$)	Global minimum weight edge connecting distinct DSU sets
Cycle Detection	Implicit (only selects edges targeting vertices outside V_T)	Explicit (via Disjoint Set Union 'FindSet' queries)
Sensitivity to Root Vertex	Execution sequence depends on initial root choice; final weight identical	Completely root-independent (global edge sort drives sequence)
Handling Duplicate Edge Weights	Determined by priority queue tie-breaking order	Determined by initial sorting algorithm tie-breaking order

5. Correctness Proofs and Algorithmic Proofs

5.1 Proof of Correctness for Prim's Algorithm

Theorem 1: *Prim's algorithm correctly constructs a Minimum Spanning Tree of any connected, weighted graph G .*

Proof (by Loop Invariant and Mathematical Induction):

We establish the following loop invariant: *At each iteration of Prim's algorithm, the set of edges E_T selected so far is a subset of some Minimum Spanning Tree T^* of G .*

- **Initialization:** Before the first iteration, $E_T = \emptyset$. An empty set is trivially a subset of any MST T^* . Thus, the invariant holds initially.
- **Maintenance:** Assume $E_T \subseteq T^*$ for some MST T^* . Let (u, v) be the next edge selected by Prim's algorithm, where $u \in V_T$ and $v \in V \setminus V_T$. By definition of the algorithm, (u, v) is a light edge crossing the cut $(V_T, V \setminus V_T)$.

Case 1: If $(u, v) \in T^*$, then $E_T \cup \{(u, v)\} \subseteq T^*$, maintaining the invariant.

Case 2: If $(u, v) \notin T^*$, since T^* is a spanning tree, there exists a unique simple path P in T^* connecting u and v . Path P must cross the cut $(V_T, V \setminus V_T)$ at least once via some edge (x, y) , where $x \in V_T$ and $y \in V \setminus V_T$.

Since (u, v) was chosen by Prim's algorithm as the minimum weight edge crossing the cut, we have:

$w(u, v) \leq w(x, y)$ Consider constructing a new spanning tree $T' = T^* \setminus \{(x, y)\} \cup \{(u, v)\}$. The weight of T' is:

$w(T') = w(T^*) - w(x, y) + w(u, v) \leq w(T^*)$ Because T^* is an MST, $w(T^*) \leq w(T')$, which implies $w(T') = w(T^*)$. Thus, T' is also a Minimum Spanning Tree, and $E_T \cup \{(u, v)\} \subseteq T'$. The invariant is maintained.

- **Termination:** The algorithm terminates when $V_T = V$, at which point $|E_T| = |V| - 1$. Since E_T forms an acyclic connected subgraph spanning all vertices, E_T is an MST. ■

5.2 Proof of Correctness for Kruskal's Algorithm

Theorem 2: *Kruskal's algorithm correctly constructs a Minimum Spanning Tree of any connected,*

weighted graph G .

Proof (by Structural Induction and Cut Property):

Let $T_k = (V, E_k)$ be the set of edges selected after k steps of Kruskal's algorithm. We prove by induction that E_k is a subset of some MST T^* .

- **Base Step ($k=0$):** $E_0 = \emptyset \subseteq T^*$ holds trivially.
- **Inductive Step:** Assume $E_k \subseteq T^*$ for an MST T^* . Let $e = (u, v)$ be the next edge chosen by Kruskal's algorithm. Edge e is the minimum-weight edge in $E \setminus E_k$ that does not form a cycle with E_k .

Therefore, u and v belong to two separate connected components in the forest $F = (V, E_k)$. Let C_u be the connected component containing u . Consider the cut $(S, V \setminus S)$ where $S = C_u$.

No edge in E_k crosses this cut because C_u is a maximal connected component in (V, E_k) . Furthermore, $e = (u, v)$ crosses this cut. Any edge e' that crosses $(S, V \setminus S)$ and is not in E_k must have $w(e') \geq w(e)$ because edges are processed in non-decreasing order of weight, and all previously examined smaller edges that could connect C_u were either already added or rejected due to cycle formation within C_u .

Thus, e is a light edge crossing the cut $(S, V \setminus S)$. By the Cut Property, e belongs to some MST containing E_k . Thus $E_{k+1} \subseteq T^*$ for some MST T^* .

- **Conclusion:** Upon selecting $|V| - 1$ edges, the result is a spanning tree of minimum weight. ■

6. Detailed Complexity & Performance Analysis

The operational efficiency of Prim's and Kruskal's algorithms varies dramatically depending on the underlying graph density and choice of data structures. Graph density is represented by the ratio of edges $m = |E|$ to vertices $n = |V|$.

6.1 Prim's Algorithm Complexity Analysis

Prim's efficiency depends on the priority queue data structure used to maintain key values:

1. Adjacency Matrix + Array Priority Queue:

- Finding minimum key: $O(n)$ performed n times $\rightarrow O(n^2)$.
- Key update (Decrease-Key): $O(1)$ performed m times $\rightarrow O(m)$.
- Total Time Complexity: $O(n^2 + m) = O(n^2)$.
- *Optimal Choice for:* Highly dense graphs where $m \approx n^2$.

2. Adjacency List + Binary Min-Heap:

- Building initial heap: $O(n)$.
- Extract-Min operations: n times $O(\log n) \rightarrow O(n \log n)$.
- Decrease-Key operations: m times $O(\log n) \rightarrow O(m \log n)$.
- Total Time Complexity: $O((n + m) \log n) = O(m \log n)$.
- *Optimal Choice for:* Sparse graphs where $m \ll n^2$.

3. Adjacency List + Fibonacci Heap:

- Extract-Min operations: n times amortized $O(\log n) \rightarrow O(n \log n)$.
- Decrease-Key operations: m times amortized $O(1) \rightarrow O(m)$.
- Total Time Complexity: $O(m + n \log n)$.
- *Theoretical Significance:* Optimal asymptotic time complexity across all densities.

6.2 Kruskal's Algorithm Complexity Analysis

Kruskal's execution time is dominated by edge sorting and Disjoint-Set Union (DSU) operations:

- 1. Edge Sorting Phase:** Sorting m edges takes $O(m \log m)$ time. Since $m \leq n^2$, $\log m \leq 2 \log n$, making sorting $O(m \log n)$.
- 2. Disjoint-Set Operations Phase:** Utilizing *Path Compression* and *Union by Rank/Size*:
 - n 'MakeSet' operations: $O(n)$.
 - At most $2m$ 'FindSet' and $n - 1$ 'UnionSet' operations.
 - Amortized time per DSU operation: $O(\alpha(n))$, where $\alpha(n)$ is the inverse Ackermann function, which grows so slowly that $\alpha(n) \leq 4$ for all practical universe scales (e.g., $n < 10^{\{600\}}$).
 - Total DSU time: $O(m \alpha(n))$.
- 3. Total Time Complexity:** $O(m \log m + m \alpha(n)) = O(m \log n)$.

6.3 Mathematical Density Comparison (Dense vs. Sparse Graphs)

Let us formally compare performance in two extreme topological configurations:

- **Sparse Graph Case ($m = O(n)$):**
 - Prim (Binary Heap): $O(n \log n)$
 - Prim (Fibonacci Heap): $O(n \log n)$
 - Kruskal (Standard Sort + DSU): $O(n \log n)$
 - *Performance Verdict:* Both algorithms perform similarly in sparse graphs. Kruskal often has a smaller constant factor due to simpler array-based DSU operations.
- **Dense Graph Case ($m = O(n^2)$):**
 - Prim (Array implementation): $O(n^2)$
 - Prim (Binary Heap): $O(n^2 \log n)$
 - Prim (Fibonacci Heap): $O(n^2)$
 - Kruskal (Sorting): $O(n^2 \log(n^2)) = O(2 n^2 \log n) = O(n^2 \log n)$
 - *Performance Verdict:* **Prim's Algorithm (Adjacency Matrix / Array) strictly outperforms Kruskal's Algorithm in dense graphs** by a factor of $O(\log n)$, because Kruskal is bogged down by sorting $O(n^2)$ edges.

Comprehensive Performance Matrix

Metric / Algorithm	Prim (Array)	Prim (Binary Heap)	Prim (Fibonacci Heap)	Kruskal (DSU + QuickSort)
Time (Sparse: $m = O(n)$)	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Time (Dense: $m = O(n^2)$)	$O(n^2)$	$O(n^2 \log n)$	$O(n^2)$	$O(n^2 \log n)$

Metric / Algorithm	Prim (Array)	Prim (Binary Heap)	Prim (Fibonacci Heap)	Kruskal (DSU + QuickSort)
Space Complexity	$O(n^2)$ or $O(n + m)$	$O(n + m)$	$O(n + m)$	$O(n + m)$
Primary Data Structure	2D Matrix / Array	Adj List + Min-Heap	Adj List + Fib-Heap	Edge Array + DSU Structure
Best Graph Topology	Dense Graphs	Sparse / General	Dense / Theoretical	Sparse / Edge-List Graphs

7. Structural Implementation Analysis & Code Templates

To highlight implementation differences, below are clean C++ code templates for both algorithms.

7.1 Implementation of Prim's Algorithm (C++ Heap Implementation)

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

typedef pair<int, int> pii; // {weight, vertex}
```



```

void primMST(int V, vector<vector<pii>>& adj) {
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    int src = 0; // Starting at vertex 0

    vector<int> key(V, 1e9);
    vector<int> parent(V, -1);
    vector<bool> inMST(V, false);

    pq.push({0, src});
    key[src] = 0;

    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        if (inMST[u]) continue;
        inMST[u] = true;

        for (auto& edge : adj[u]) {
            int v = edge.first;
            int weight = edge.second;

            if (!inMST[v] && key[v] > weight) {
                key[v] = weight;
                pq.push({key[v], v});
                parent[v] = u;
            }
        }
    }

    int totalWeight = 0;
    cout << "Prim's MST Edges:\n";
    for (int i = 1; i < V; ++i) {
        cout << parent[i] << " - " << i << " | Weight: " << key[i] << "\n";
        totalWeight += key[i];
    }
}

```

```

    cout << "Total MST Weight: " << totalWeight << "\n";
}

```

7.2 Implementation of Kruskal's Algorithm (C++ DSU Implementation)

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Edge {
    int u, v, weight;
    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};

class DisjointSet {
    vector<int> parent, rank;
public:
    DisjointSet(int n) {
        parent.resize(n);
        rank.assign(n, 0);
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int i) {
        if (parent[i] == i)
            return i;
        return parent[i] = find(parent[i]); // Path compression
    }

    bool unite(int i, int j) {
        int root_i = find(i);
        int root_j = find(j);

```

```

        if (root_i != root_j) {
            // Union by rank
            if (rank[root_i] < rank[root_j]) swap(root_i, root_j);
            parent[root_j] = root_i;
            if (rank[root_i] == rank[root_j]) rank[root_i]++;
            return true;
        }
        return false;
    }
};

void kruskalMST(int V, vector<Edge>& edges) {
    sort(edges.begin(), edges.end());
    DisjointSet dsu(V);

    int totalWeight = 0;
    int edgesCount = 0;
    cout << "Kruskal's MST Edges:\n";

    for (const auto& edge : edges) {
        if (dsu.unite(edge.u, edge.v)) {
            cout << edge.u << " - " << edge.v << " | Weight: " << edge.weight << "\n";
            totalWeight += edge.weight;
            edgesCount++;
            if (edgesCount == V - 1) break;
        }
    }

    cout << "Total MST Weight: " << totalWeight << "\n";
}

```

8. Contextual Suitability & Application Guidelines

Choosing between Prim's and Kruskal's algorithms in software engineering and system architecture depends

on graph characteristics and operational constraints:

8.1 Practical Selection Criteria

- **Choose Prim's Algorithm when:**
 1. The graph is **dense** (i.e., $m \approx O(n^2)$). Using an adjacency matrix with an array-based Prim implementation yields optimal $O(n^2)$ execution without sorting overhead.
 2. The graph is supplied or generated dynamically via **adjacency lists** or streaming APIs centered on vertex adjacencies.
 3. Memory footprint is critical for vertices, and priority keys can be updated in place.
- **Choose Kruskal's Algorithm when:**
 1. The graph is **sparse** (i.e., $m = O(n)$), where edge sorting is fast and DSU operations run in near-linear time.
 2. The graph representation is already an **edge list** or edges are pre-sorted (or easily sortable via linear-time techniques like Counting Sort or Radix Sort for integer weights, yielding $O(m + n)$ execution).
 3. The problem requires tracking **connected components dynamically** during edge addition (e.g., Image Segmentation using Felzenszwalb's algorithm).
 4. **Clustering Applications:** Kruskal's intermediate state after adding $n - k$ edges directly yields the optimal single-linkage **k-clustering**.

8.2 Summary Synthesis Matrix

Dimension	Prim's Algorithm	Kruskal's Algorithm
Algorithmic Paradigm	Greedy (Cut-based expansion)	Greedy (Edge sorting + Cycle avoidance)

Dimension	Prim's Algorithm	Kruskal's Algorithm
Optimal Graph Density	Dense Graphs ($m \approx n^2$)	Sparse Graphs ($m \approx n$)
Time Complexity (Optimal Heap/DSU)	$O(m + n \log n)$ [Fibonacci] / $O(n^2)$ [Array]	$O(m \log n)$ [Sorting dominated]
Space Complexity	$O(V + E)$ or $O(V^2)$	$O(V + E)$
Data Structure Overhead	Priority Queue (Min-Heap / Fib-Heap)	Disjoint-Set Union (DSU) + Sorting
Parallelization Potential	Low (sequential single-tree growth)	High (edges can be filtered/sorted in parallel)

9. Conclusion

Both Prim's and Kruskal's algorithms guarantee finding an optimal Minimum Spanning Tree for connected, weighted undirected graphs. However, their structural mechanisms differ fundamentally. Prim's algorithm grows a single connected tree vertex by vertex using local cut decisions, making it well suited for dense graphs. Kruskal's algorithm builds a spanning forest by selecting global light edges while avoiding cycles via Disjoint Set Union operations, making it well suited for sparse graphs and clustering tasks. Understanding these performance tradeoffs ensures the optimal algorithm is selected for a given application.